UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 0. Real-bank slots: 11. Unofficial model answers:

   11/11.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (2).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) An interrupt is an asynchronous signal or event, generated by hardware (e.g. an I/O device completing a transfer, a timer, or an error) or by software (a trap), that causes the CPU to suspend the program it is currently executing, save its context (PC and register state), and transfer control to a special service routine — the interrupt handler or interrupt service routine (ISR). When the handler finishes, the saved context is restored and the interrupted program resumes, so the interrupt is transparent to the running program. Interrupts allow the CPU to respond promptly to external events instead of polling.

(b) Two ways to handle multiple interrupts:

1. Sequential (disabled/priority-ordered) handling: when an interrupt is being serviced, further interrupts are disabled (or masked) until the current ISR completes. Pending interrupts are then examined in a fixed priority order and serviced one at a time. This is simple and avoids nesting/race problems, but urgent interrupts may have to wait for lower-priority ones to finish, increasing latency.

2. Nested (priority-based, interruptible) handling: a higher-priority interrupt can preempt the ISR of a lower-priority interrupt. The CPU compares the priority of the new request with that of the interrupt currently being serviced; if higher, the current ISR's context is saved and the higher-priority ISR runs first, then control returns to the interrupted lower-priority ISR. Lower- or equal-priority requests are held pending. This gives fast response to critical events but requires priority hardware and careful saving/restoring of nested contexts.

(Other acceptable pairs: polling/software priority vs. vectored interrupt with hardware priority; masking vs. daisy-chaining.)

Unofficial marking guide:

- *1 mark: correct definition of an interrupt (asynchronous event/signal that suspends the current program, saves context, and transfers control to an ISR, resuming afterwards).*
- *2 marks: first method of handling multiple interrupts clearly described (e.g. sequential/disabled handling with masking and priority order).*
- *2 marks: second method clearly described (e.g. nested/priority-based preemptive handling), including how priority decides service order.*

Model confidence: high.

Model note: Figure 1 (instruction-cycle state diagram with interrupt) is not needed to answer the subparts; only its extracted text was available, so no figure detail is relied upon. The question text contains OCR noise ('cfi', 'tc', 're', etc.) which does not affect the answer.

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (8).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The personal computer was introduced in the fourth generation of computers (approximately 1971–present), which is characterised by microprocessors and very-large-scale integration (VLSI). The Altair 8800 (1975) mentioned in the snippet is a classic fourth-generation personal computer.

(b) One other major discovery of the fourth generation is the microprocessor, exemplified by the Intel 4004 (1971). The microprocessor integrates the CPU onto a single silicon chip using VLSI technology, which made possible the development of affordable personal computers, and also enabled other advances such as graphical user interfaces (GUIs) and local-area networks (LANs).

(c) Advances in laptops and hand-held devices are feasible in the fourth generation because of continued miniaturisation and integration: VLSI and ULSI allow millions of transistors on a single chip, reducing size, power consumption and cost while increasing speed and reliability. Compared with previous generations that used discrete transistors or small-scale integrated circuits, fourth-generation devices use microprocessors and high-density memory chips, making portable, battery-powered computing practical.

© Hak Cipta Universiti Teknologi MARA

Unofficial marking guide:

• 1 mark: (a) Correctly identifies the fourth generation (microprocessor generation) as the generation in which the personal computer was introduced.

• 2 marks: (b) Clearly discusses one other major discovery of the fourth generation (e.g., the microprocessor) and explains its significance (1 mark for naming the discovery, 1 mark for explaining its impact).

• 2 marks: (c) Explains how VLSI/ULSI and microprocessor technology enable miniaturisation, low power and low cost for laptops/hand-held devices (1 mark), and contrasts this with previous generations that used discrete components or SSI/MSI circuits (1 mark).

Model confidence: high.

Model note: The question text contains a minor OCR/grammar error ('that a personal computer is introduced'); interpreted as 'in which generation the personal computer was introduced'. The snippet mentions the Altair 8800, which is a fourth-generation machine. The answer assumes standard generation classification used in CSC429.

QUESTION A3 (5 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (5).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A mapping function is the rule that determines where a block of main memory is placed in the cache — i.e. it specifies the correspondence between main-memory block addresses and cache line (slot) addresses. Common mapping functions are direct mapping, fully associative mapping and set-associative mapping.

(b) Direct mapping: each main-memory block maps to exactly one fixed cache line, given by (block address) mod (number of cache lines). The tag is stored with the line and compared on every access; only one comparison is needed, the hardware is simple and cheap, but conflict misses are high because many blocks compete for the same line. Fully associative mapping: a block may be placed in any cache line; the whole block address is stored as the tag and must be compared against every line's tag in parallel (using a comparator per line). It is flexible with the lowest conflict-miss rate, but needs expensive parallel comparison hardware and a replacement policy.

(c) Address = 28 bits, word size = 4 bits, line size = 10 bits. Number of words per line = line size / word size = 10/4 = 2.5 — not an integer, so the figures are inconsistent; taking the standard interpretation that the line (block) offset field is 10 bits (i.e. 2^10 addressable units per line) and the word offset within a word is 4 bits: Direct mapping: offset = word offset + line offset = 4 + 10 = 14 bits, so tag = 28 − 14 = 14 bits. Associative mapping: there is no line-index field, so tag = 28 − 10 = 18 bits (or, if the 4-bit word offset is also removed, 28 − 14 = 14 bits). Under the literal reading (word size 4 bits, line 10 bits, words/line = 2.5) the split is not valid, so the 10-bit line-offset interpretation is used.

Unofficial marking guide:

• 1 mark: correct definition of mapping function as the rule placing main-memory blocks into cache lines, with mention of the mapping types.

• 2 marks: 1 mark for correct description of direct mapping (fixed line = block mod lines, single tag comparison, conflict misses); 1 mark for correct description of associative mapping (any line, full tag compared in parallel, flexible but costly hardware).

• 2 marks: 1 mark for correct offset/tag arithmetic for direct mapping (tag = 28 − (10 + 4) = 14 bits); 1 mark for correct associative tag (28 − 10 = 18 bits, or 14 bits if word offset also removed), with the inconsistency of 10/4 noted.

Model confidence: medium.

Model note: The question's 'word size = 4 bit, line = 10 bit' gives 2.5 words per line, which is not an integer; the answer assumes 'line = 10 bit' means a 10-bit line/block offset and 'word size = 4 bit' means a 4-bit word offset, and states the alternative associative result. If the intended reading was line size = 10 bits total with 4-bit words, the tag values differ; the figure/table (if any) should be checked.

QUESTION A4 (5 marks)

Topic: Data representation and number systems

Source: CSC429 (11).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A byte is a group of 8 bits (binary digits) treated as a single unit of data. It is the standard addressable unit of memory in most computers and can represent 2^8 = 256 distinct values (e.g. 0–255 unsigned, or one ASCII character).

(b) Base 10 is not suitable for computer number representation because digital hardware is built from two-state (bistable) devices — transistors/switches that reliably distinguish only two states, ON/OFF or

© Hak Cipta Universiti Teknologi MARA

high/low voltage. These map naturally onto the two binary digits 0 and 1, so base 2 gives simple, reliable, low-cost storage and logic. Base 10 would require ten distinct, reliably distinguishable physical states per digit, which is far harder to build, more error-prone and more expensive; arithmetic and logic circuits (adders, gates) are also simplest in binary. Hence computers use base 2 (with octal/hex only as convenient shorthand for humans).

(c) In IEEE 754 single precision the exponent field is 8 bits and is stored in biased (excess) form with a bias of 127, i.e. stored value = true exponent + 127. The 8-bit field can hold 0–255, but the all-zeros pattern (0) and the all-ones pattern (255) are reserved for special values (zero/denormals and infinity/NaN respectively). The usable stored range is therefore 1–254, giving true exponents from 1−127 = −126 up to 254−127 = +127. So the maximum representable (normal) exponent is 127, not 128: 128 would require a stored value of 255, which is reserved. (Equivalently, reserving the two end patterns costs one value at the top, capping the exponent at 127.)

Unofficial marking guide:

• 1 mark: byte = 8 bits (a group of 8 binary digits treated as one unit)
• 1 mark: hardware is two-state/bistable, naturally matching binary 0/1
• 1 mark: base 10 would need ten distinguishable states — unreliable/expensive/complex, so binary is used
• 1 mark: exponent is 8-bit biased by 127; stored value = exponent + 127
• 1 mark: all-zeros and all-ones (0 and 255) reserved for special values, so usable stored range 1–254 gives max true exponent 254−127 = 127 (128 would need reserved 255)
Model confidence: high.
Model note: Part (c) is interpreted for IEEE 754 single precision (8-bit exponent, bias 127); the question's wording matches this standard case. No figure required.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (17).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The icon in Figure 1 is the USB (Universal Serial Bus) trident/arrow logo, so the interface is a USB interface — a serial, plug-and-play external device interface used to connect peripherals such as keyboards, mice, printers and storage devices to the computer.

(b) A hot connection (hot plugging / hot swapping) means a device can be connected to or disconnected from the running system while the power is on, without switching off or rebooting the computer. The interface and operating system detect the device automatically, configure it (plug-and-play) and load the required driver, so the connection is made safely 'live'.

(c) The three I/O techniques are:
1. Programmed I/O (polled I/O): The CPU issues the I/O command and then repeatedly checks (polls) the device status register until the device is ready, then transfers the data itself. CPU involvement is very high — the CPU is fully occupied waiting and moving data, wasting cycles (busy-waiting).
2. Interrupt-driven I/O: The CPU starts the device and continues with other work. When the device is ready it raises an interrupt; the CPU suspends its current program, saves context, runs the interrupt service routine to transfer the data, then resumes. CPU involvement is reduced — the CPU is only involved when the interrupt occurs, though it still performs the data transfer.
3. Direct Memory Access (DMA): The CPU programs a DMA controller with the source/destination and count, then the DMA controller transfers data directly between the device and memory without CPU involvement. The CPU is interrupted only once, when the whole block transfer is complete, so CPU involvement is minimal and it can do other work during the transfer.

Unofficial marking guide:

• 1 mark: identifies the interface as USB (Universal Serial Bus)
• 1 mark: brief description of USB as a serial/plug-and-play external device interface
• 1 mark: defines hot connection as connecting/disconnecting a device while the system is powered on
• 1 mark: notes automatic detection/plug-and-play and no reboot required
• 1 mark: names programmed I/O and explains CPU polls/busy-waits and transfers data itself
• 1 mark: names interrupt-driven I/O and explains device interrupts CPU, CPU runs ISR to transfer data
• 1 mark: names DMA and explains DMA controller transfers data directly to/from memory
• 1 mark: explains CPU involvement is minimal in DMA (interrupted only at completion)
• 1 mark: compares/elaborates CPU involvement across the three techniques (high → reduced → minimal)
• 1 mark: overall clarity and correct terminology (status register, interrupt service routine, DMA controller)

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.

Model note: Figure 1 is only available as extracted text; the answer assumes the icon is the standard USB trident logo, which is the usual interpretation for this question. If the actual icon differs, part (a) and the hot-connection link in (b) should be re-checked against the figure.

QUESTION A6 (10 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (21).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Completing the taxonomy (Figure 4). The standard taxonomy (Flynn-style, as drawn in Stallings) branches as follows. Top level: Processor Organization splits into Single Instruction and Multiple Instruction. Under Single Instruction the two children are Single Data (SISD) and Multiple Data (SIMD). Under Multiple Instruction the child is Multiple Data (MIMD). The SIMD branch leads to a Uniprocessor (a single processor with SIMD-style data parallelism). The MIMD branch splits into Distributed Memory and Shared Memory. Distributed Memory gives (b) = Multiprocessor (loosely coupled / cluster-type, each processor with its own memory). Shared Memory splits into Symmetric (SMP) and Non-Uniform Memory Access (NUMA). Therefore the missing labels are: (a) = Single Data (SISD); (b) = Distributed-memory multiprocessor (loosely coupled multiprocessor); (c) = Shared Memory (tightly coupled multiprocessor); (d) = Multiple Instruction, Multiple Data (MIMD) — the node that fans out to Distributed/Shared memory; (e) = Shared-memory multiprocessor (tightly coupled); (f) = Symmetric Multiprocessor (SMP); (g) = NUMA (Non-Uniform Memory Access). In short: (a) SISD, (b) Distributed-memory multiprocessor, (c) Shared memory, (d) MIMD, (e) Shared-memory multiprocessor, (f) SMP, (g) NUMA.

(b) Two disadvantages of SMP. (1) Memory contention / limited scalability: all processors share one common memory and a single system bus or interconnect, so as more processors are added they compete for the same memory bandwidth and bus; this causes contention, longer memory access latency and diminishing returns, capping practical scalability (typically a small number of processors). (2) Cache coherence overhead and software complexity: because each processor has its own cache but shares main memory, hardware must keep caches coherent (snooping or directory protocols), which adds traffic and complexity; writing correct, efficient parallel software (locking, synchronization, avoiding false sharing) is difficult, and a single shared OS/kernel can become a bottleneck and a single point of failure.

Unofficial marking guide:

- *1 mark: (a) correctly identified as Single Data (SISD)*
- *1 mark: (b) correctly identified as Distributed-memory (loosely coupled) multiprocessor*
- *1 mark: (c) correctly identified as Shared Memory*
- *1 mark: (d) correctly identified as MIMD (Multiple Instruction Multiple Data)*
- *1 mark: (e) correctly identified as Shared-memory (tightly coupled) multiprocessor*
- *1 mark: (f) and (g) correctly identified as Symmetric (SMP) and NUMA respectively*
- *2 marks: first SMP disadvantage (memory/bus contention and limited scalability) explained*
- *2 marks: second SMP disadvantage (cache-coherence overhead / software and OS complexity) explained*

Model confidence: medium.

Model note: Figure 4 is only available as extracted text; the exact letter positions (a)-(g) are inferred from the standard Stallings taxonomy. Labels (d) and (e) are ambiguous in the OCR (the MIMD node and the shared-memory multiprocessor node); the answer assigns them to the most consistent reading. Accept equivalent wording (e.g. 'loosely coupled' for distributed, 'tightly coupled' for shared memory).

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (1).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Interrupt-driven I/O is an input/output technique in which the processor initiates an I/O operation and then continues executing other instructions; the I/O device or its controller sends an interrupt request signal to the processor when it is ready to transfer data or has completed the operation. The processor responds by suspending its current program, saving the necessary context, and branching to an interrupt service routine (ISR) that services the device (e.g., transfers a data word). After the ISR, the saved context is restored and the interrupted program resumes. Thus the processor does not repeatedly poll the device status; it is notified by the interrupt.

© Hak Cipta Universiti Teknologi MARA

(b) When the processor, main memory and I/O share a common bus, two modes of addressing are:
1. Memory-mapped I/O: I/O device registers are assigned addresses within the same address space as main memory. The processor uses the same load/store instructions and the same address lines to access either memory or an I/O register; the address decoder selects memory or the I/O device according to the address range. No special I/O instructions are needed, but a portion of the memory address space is consumed by I/O.
2. Isolated (port-mapped) I/O: I/O devices have a separate address space distinct from main memory. The processor uses special I/O instructions (e.g., IN/OUT) and a separate control signal (e.g., M/IO) to indicate whether the address on the bus refers to memory or to an I/O port. This preserves the full memory address space but requires additional instructions and control lines.

(c) Advantage: The processor does not waste time polling the device; it can execute other useful work while the I/O operation proceeds, and it is only interrupted when service is needed. This improves CPU utilization and system throughput, especially when I/O devices are slow or infrequent.
Disadvantage: Interrupt-driven I/O requires additional hardware (interrupt controller, priority logic) and software overhead (context save/restore, ISR execution). Each interrupt involves saving and restoring processor state, which adds overhead; frequent interrupts can degrade performance, and the design is more complex than programmed I/O.

Unofficial marking guide:

• 1 mark: states that the processor initiates I/O and continues other work, and the device interrupts when ready (definition of interrupt-driven I/O).

• 1 mark: explains that the processor suspends current program, saves context, and branches to an ISR to service the device.

• 2 marks: describes memory-mapped I/O correctly (shared address space, same instructions, address decoder distinguishes memory vs I/O).

• 2 marks: describes isolated/port-mapped I/O correctly (separate address space, special I/O instructions, separate control signal).

• 2 marks: gives one valid advantage with explanation (e.g., no polling, CPU can do other work, improved efficiency/throughput).

• 2 marks: gives one valid disadvantage with explanation (e.g., hardware/software overhead, context save/restore, complexity, frequent interrupts degrade performance).

Model confidence: high.

Model note: The question text contains OCR errors: '1/0' is interpreted as 'I/O'. The term 'modes of addressing' is taken to mean methods of addressing I/O devices when sharing a common bus; memory-mapped and isolated I/O are the standard two modes. No figure or table is required.

QUESTION A8 (10 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (3).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Four classifications of multiple processor architecture: (1) Symmetric Multiprocessor (SMP) – all processors share a single main memory and have equal access to I/O, with a single OS image. (2) Non-Uniform Memory Access (NUMA) – multiple processors with local memories; access time to a memory location depends on which processor owns it. (3) Cluster – a collection of independent computers (nodes) connected by a network, working together as a single system. (4) Massively Parallel Processor (MPP) – many processors each with their own private memory, connected by a high-speed interconnect. (Other acceptable classifications include: vector processor, shared-memory multiprocessor, distributed-memory multiprocessor, and multicore.)

(b) X is MIMD (Multiple Instruction, Multiple Data) – the multiple processor architecture class in Flynn's taxonomy that includes SMP, NUMA and clusters.

(c) Differentiation: SMP has a single shared main memory accessed uniformly by all processors; all processors are under one operating system and are tightly coupled. A cluster consists of separate computers (nodes), each with its own processor and local memory, connected by a network; it is loosely coupled and each node runs its own OS instance. In SMP, communication between processors is via shared memory (fast); in a cluster, communication is via message passing over the network (slower). SMP scales poorly beyond a small number of processors due to memory contention; clusters scale well by adding nodes. SMP is easier to manage and control: it presents a single system image, one OS manages all resources, and there is no need to distribute and coordinate work across separate machines. Clusters require middleware, distributed scheduling, and handling of network failures, making

© Hak Cipta Universiti Teknologi MARA

administration more complex.

Unofficial marking guide:

• 1 mark: Correctly listing SMP (Symmetric Multiprocessor) as a classification.
• 1 mark: Correctly listing NUMA (Non-Uniform Memory Access) as a classification.
• 1 mark: Correctly listing Cluster as a classification.
• 1 mark: Correctly listing MPP (Massively Parallel Processor) or another valid classification (e.g., vector processor, shared-memory multiprocessor).
• 2 marks: Identifying X as MIMD (Multiple Instruction, Multiple Data) – 2 marks for correct identification; 1 mark if only 'multiple processor' or vague answer.
• 1 mark: Differentiating SMP and Cluster in terms of memory/communication (e.g., SMP uses shared memory; cluster uses message passing over network).
• 1 mark: Differentiating SMP and Cluster in terms of coupling/OS (e.g., SMP is tightly coupled with single OS; cluster is loosely coupled with separate OS per node).
• 1 mark: Stating opinion that SMP is easier to manage and control.
• 1 mark: Providing appropriate justification (e.g., single system image, one OS, no distributed coordination, simpler administration).
Model confidence: high.
Model note: Assumption: 'Symetric' is a typo for 'Symmetric'. The question asks for 'classification of multiple processor architecture'; the answer uses common categories (SMP, NUMA, Cluster, MPP). Other valid classifications are accepted as noted. Part (b) assumes X refers to Flynn's taxonomy class MIMD, as SMP, NUMA and clusters are all MIMD architectures.

QUESTION A9 (10 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (18).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Identifying the concept — Computer Architecture vs Computer Organization:

- "Deals with the logical aspects of the system, including instruction sets, data formats, and addressing modes" → Computer Architecture. Architecture is the programmer-visible, logical view: it specifies the instruction set, the data formats the machine supports, and the addressing modes available to instructions.
- "Defines the functionality, capabilities, and programming model" → Computer Architecture. Architecture answers *what* the machine does and how it appears to software (the programming model), independent of how it is built.
- "Concerned with how the computer will execute the tasks and implement the functions" → Computer Organization. Organization is the implementation view: it decides *how* the architectural functions are realised in hardware.
- "Involves the hardware layout, circuits, control signals, memory access, and timing" → Computer Organization. Organization covers the physical/structural details — datapath and circuit layout, control-signal generation, memory-access mechanisms and timing.

Summary: the first two scenarios describe architecture (logical, functional, programmer-visible); the last two describe organization (physical implementation, control and timing).

(b) Three CPU registers involved in data processing (arithmetic operations):

1. Accumulator (AC). A general-purpose working register that holds one operand before an arithmetic/logic operation and receives the result afterwards. In a single-accumulator machine, e.g. ADD X computes AC ← AC + M[X], so the accumulator supplies the implicit first operand and stores the sum. It is directly connected to the ALU, minimising data movement for repeated arithmetic.

2. General-purpose registers (GPRs), e.g. R0–R15. A small, fast register file that holds operands and intermediate results for arithmetic instructions. Register-to-register instructions (e.g. ADD R1, R2, R3 → R1 ← R2 + R3) avoid slow memory accesses, and GPRs also hold addresses and loop counters. They are the main source/destination of ALU operands in load-store and RISC-style machines.

3. Program Counter (PC) / Instruction Register (IR) — fetch-side registers that feed processing. The PC holds the address of the next instruction to fetch; the IR holds the instruction currently being decoded/executed. Together they sequence the fetch–decode–execute cycle so that the correct arithmetic instruction and its operands are delivered to the ALU. (Either the PC or the IR may be described; both are valid processing-cycle registers.)

Other acceptable registers: Status/Flag register (PSW) — holds condition flags (Zero, Carry, Sign,

© Hak Cipta Universiti Teknologi MARA

Overflow) produced by arithmetic operations and used for conditional branches; Memory Address Register (MAR) and Memory Data Register (MDR) — hold the address and data during operand fetch/store for arithmetic instructions.

Unofficial marking guide:

• 1 mark: correctly identifies 'logical aspects — instruction sets, data formats, addressing modes' as Computer Architecture

• 1 mark: correctly identifies 'functionality, capabilities, programming model' as Computer Architecture

• 1 mark: correctly identifies 'how the computer executes/ implements functions' as Computer Organization

• 1 mark: correctly identifies 'hardware layout, circuits, control signals, memory access, timing' as Computer Organization

• 2 marks: names and describes the Accumulator (holds operand/result of ALU operations)

• 2 marks: names and describes a general-purpose register file (holds operands/intermediate results for arithmetic)

• 2 marks: names and describes a third processing register (PC/IR or Status/Flag register) with its role in data processing

Model confidence: high.

Model note: The question's table is given only as extracted text; the four scenario rows are matched in order to the four concept cells. Part (b) asks for 'THREE' registers; the model answer gives three with a note that PC/IR or the flag register are interchangeable as the third, and lists other acceptable registers. This is an unofficial model answer, not an official marking scheme.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

Source: CSC429 (10).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Binary is used as the machine language because digital hardware is built from two-state devices (transistors/switches) that are naturally ON or OFF, so two symbols (0 and 1) map directly onto physical states with high noise immunity and simple, cheap, reliable circuits. Binary also suits Boolean algebra, so logic gates and arithmetic units can be designed directly from it, and it gives a uniform representation for numbers, characters and instructions. [2 marks]

(b) Working: ACEh = 10×256 + 12×16 + 14 = 2560 + 192 + 14 = 2766 (decimal). BFh = 11×16 + 15 = 176 + 15 = 191 (decimal). Difference = 2766 − 191 = 2575 (decimal).

(i) Hexadecimal: ACEh − BFh. Borrow in hex (base 16): E − F requires a borrow: (14+16) − 15 = 15 = F. Next column: C was borrowed from, so it is B; B − B = 0. Next column: A − 0 = A. Result = A0Fh. Check: A0Fh = 10×256 + 0 + 15 = 2575. ✓

(ii) Octal: Convert 2575 decimal to octal. 2575 ÷ 8 = 321 r 7; 321 ÷ 8 = 40 r 1; 40 ÷ 8 = 5 r 0; 5 ÷ 8 = 0 r 5. Reading remainders upward: 5017_8. Check: 5×512 + 0×64 + 1×8 + 7 = 2560 + 0 + 8 + 7 = 2575. ✓

(iii) Binary: Convert 2575 decimal to binary (or convert A0Fh directly). A0Fh = 1010 0000 1111_2. Check: 1010 0000 1111_2 = 2048 + 512 + 8 + 4 + 2 + 1 = 2575. ✓ (Equivalently 5017_8 = 101 000 001 111_2.)

Unofficial marking guide:

• 1 mark: states that digital hardware has two stable states (ON/OFF, high/low) and binary maps directly onto them

• 1 mark: explains that binary enables simple, reliable Boolean-logic/arithmetic circuits and uniform encoding of data and instructions

• 1 mark: correct hexadecimal subtraction ACEh − BFh = A0Fh with borrow shown

• 1 mark: correct hexadecimal result A0Fh (or equivalent decimal 2575 shown as intermediate)

• 1 mark: correct conversion of the result to octal (5017_8) with method shown

• 1 mark: correct octal digits 5017

• 1 mark: correct conversion of the result to binary (1010 0000 1111_2) with method shown

• 1 mark: correct binary digits 101000001111

Model confidence: high.

Model note: Assumes ACEh and BFh are unsigned hexadecimal values and that part (b) asks for the single difference expressed in each base. OCR spacing ('ACEh- BFh') read as ACEh − BFh. No figure or table required.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (14).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation: Figure 4 is read as z = A + B² − C(D − C/B), i.e. B2 is taken as B squared (B^2). The expression is evaluated in the order: t1 = C/B; t2 = D − t1; t3 = C × t2; t4 = B × B; t5 = A + t4; z = t5 − t3. Registers A, B, C, D are read-only; x and y are scratch; the result is left in z.

Zero-address program (stack machine). Each operand is pushed, the operation consumes the top two stack entries and pushes the result; POP stores the top of stack into the named register.

```
; z = A + B^2 - C*(D - C/B)
PUSH C      ; stack: C
PUSH B      ; stack: C, B
DIV         ; stack: C/B           (t1)
PUSH D      ; stack: t1, D
SUB         ; stack: D - C/B       (t2)
PUSH C      ; stack: t2, C
MUL         ; stack: C*(D - C/B)   (t3)
PUSH B      ; stack: t3, B
PUSH B      ; stack: t3, B, B
MUL         ; stack: t3, B^2       (t4)
PUSH A      ; stack: t3, t4, A
ADD         ; stack: t3, A + B^2   (t5)
SUB         ; stack: (A + B^2) - t3
POP z       ; z = A + B^2 - C*(D - C/B)
```

One-address program (accumulator machine). The accumulator holds one operand; LOADx copies x into the accumulator, OPx applies the operation between the accumulator and x, and STORx writes the accumulator to x.

```
; z = A + B^2 - C*(D - C/B)
LOAD C      ; ACC = C
DIV B       ; ACC = C/B            (t1)
STOR x      ; x = C/B
LOAD D      ; ACC = D
SUB x       ; ACC = D - C/B        (t2)
STOR x      ; x = D - C/B
LOAD C      ; ACC = C
MUL x       ; ACC = C*(D - C/B)    (t3)
STOR x      ; x = t3
LOAD B      ; ACC = B
MUL B       ; ACC = B^2            (t4)
ADD A       ; ACC = A + B^2        (t5)
SUB x       ; ACC = (A + B^2) - t3
STOR z      ; z = final answer
```

Two-address program (register/memory-register machine). MOVE(x,y) copies y into x; OP(x,y) performs x = x OP y, so the destination register is overwritten and must be chosen carefully because A, B, C, D are read-only.

```
; z = A + B^2 - C*(D - C/B)
MOVE x, C   ; x = C
DIV  x, B   ; x = C/B              (t1)
MOVE y, D   ; y = D
SUB  y, x   ; y = D - C/B          (t2)
MOVE x, C   ; x = C
MUL  x, y   ; x = C*(D - C/B)      (t3)
MOVE y, B   ; y = B
MUL  y, B   ; y = B^2              (t4)
```

© Hak Cipta Universiti Teknologi MARA

```
MOVE z, A   ; z = A
ADD  z, y   ; z = A + B^2        (t5)
SUB  z, x   ; z = (A + B^2) - t3  = final answer
```

In all three versions the final result is held in z, and A, B, C, D are never used as destinations, satisfying the constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct PUSH/POP/ADD/SUB/MUL/DIV sequence, correct operator order for C/B, D−C/B, C×(…), B×B, A+B², final subtraction, and POP z

• 4 marks: one-address program — correct LOAD/STOR/OPx sequence with accumulator handling and correct use of x as temporary

• 4 marks: two-address program — correct MOVE/OP(x,y) sequence with correct destination choice and no modification of A, B, C, D

• 3 marks: correct evaluation order/parenthesisation of z = A + B² − C(D − C/B) in all three programs (division before subtraction, multiplication before final subtraction)

• 3 marks: correct assembly notation, comments and final result stored in z in each program

• 2 marks: no read-only register (A, B, C, D) is overwritten and only x, y, z are used as temporaries

Model confidence: medium.

Model note: Figure 4 is only available as extracted text; 'B 2' is assumed to mean B squared (B^2). If the original figure shows B×2 instead, replace each B×B / MUL B step with a multiply by constant 2. Table 3 OCR is imperfect ('T\vr.-address' = two-address); instruction mnemonics are interpreted conventionally: zero-address PUSH/POP/ADD/SUB/MUL/DIV, one-address LOADx/STORx/ADDx/SUBx/MULx/DIVx, two-address MOVE(x,y)/ADD(x,y)/SUB(x,y)/MUL(x,y)/DIV(x,y). This is an unofficial model answer, not a marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA